LANGUAGES AND AUTOMATA II

Practice

Reinforcement Questions

Do you like spaced repetition learning? Have you used Anki or Quizlet? Whether or not spaced repetition works, or works for you, periodically working on flash-card like questions can be a lot of fun, and just may help you retain information. Here are a few problems tied to the course material. Visit them periodically!

Computing History

What were some of the earliest algorithms? HIDE ANSWER

- The ones on the 4000 year-old Babylonian tables
- The Rhind Papyrus
- Euclid's GCD
- Sieve of Erathostenes

From whose name do we get the word "algorithm" from, and what was one his greatest works? HIDE ANSWER

Muḥammad ibn Mūsā al-Khwārizmī, Al-Jabr

What are some computing machines that are over 2000 years old? HIDE ANSWER

- Abacus
- Astrolabe
- Antikythera Mechanism

What were Charles Babbage's two great machines? Which of the two were actually built? HIDE ANSWER

- Difference Engine (built)

- Analytical Engine (neve built)

What is Ada Lovelace, a collaborator of Babbage, known for? HIDE ANSWER

- Authoring the first program for a general-purpose computer
- Her published insights into the generality of the Analytical Engine in applications beyond arithmetic, such as the composition of musical scores

Who is the guy most associated with the formalist movement in mathematics, so much so that he felt formal systems could capture all of mathematics through proof and computation? HIDE ANSWER

David Hilbert

What were the three major attempts at formalizing computation in the 1930s? HIDE ANSWER

- Alonzo Church's Lambda Calculus
- Kurt Gödel's Mu-Recursive Functions
- Alan Turing's Turing Machines

What were the major contributions of Turing's 1936 paper? HIDE ANSWER

- A formalization of computation that arguably lined up with our intuitive sense of what computation actually is
- The demonstration of computational universality: we need only ONE machine to perform all tasks that are computable
- The demonstration of uncomputable functions

Theories

What exactly is a **theory**, in the context of art and science? HIDE ANSWER

An organized body of knowledge with explanatory and predictive powers.

Why are theories important? HIDE ANSWER

They provide us with a vocabulary to communicate ideas, test hypotheses, and generate new knowledge.

What are the four major computation theories and what are they concerned with? HIDE ANSWER

1. Language Theory: how computations are expressed
2. Automata Theory: how computations are carried out
3. Computability Theory: what can and cannot be computed
4. Complexity Theory: the resources required for certain computations

What is a language, in formal language theory? HIDE ANSWER

A set of strings over some finite alphabet.

What is the lambda calculus seemingly most concerned with representing and manipulating? HIDE ANSWER

Functions

Who came up with the idea of formalizing computing via the Lambda Calculus? HIDE ANSWER

Alonzo Church

Who came up with the idea of formalizing computing via the Turing Machine? HIDE ANSWER

Alan Turing

What was Turing trying to do when he realized he would have to do no less than formally define the very notion of an algorithm (or computational process)? HIDE ANSWER

He was trying to show Hilbert's Entscheidungsproblem did not have a solution

On what did Turing more-or-less base his fundamental idea of how a formal (abstract mechanical) computing device should work? HIDE ANSWER

(Human) computers, calculating on paper by moving through mental states looking at symbols, adding new symbols, and erasing existing ones.

What are the main components of a Turing Machine? HIDE ANSWER

1. A control unit made up of states and a transition relation, and
2. An infinite one-dimensional tape with cells that can each hold a single symbol or be blank.

Usually, a Turing machine just rewrites its input into an output. But sometimes, all we want to compute is the answer to a YES-NO question. In this case, how does the Turing Machine announce its answer? HIDE ANSWER

It says YES by entering an ACCEPT state with no outgoing transitions, and answers NO by getting into a non-ACCEPT state with no outgoing transition for the current symbol. It is possible for the machine to loop forever on a given input, in which case it is said to give no answer.

Answering a YES-NO question on a Turing Machine is isomorphic to _____ a language. HIDE ANSWER

Recognizing

Why is Turing's discovery/invention of the Universal Turing Machine so profound? HIDE ANSWER

It means that truly general-purpose, programmable computing machines exist (i.e., we don't need specialized machines for all computations).

How can we prove there are functions that cannot be computed? HIDE ANSWER

By diagonalizing over an assumed enumeration of all functions.

What does the Church-Turing Thesis say? HIDE ANSWER

That the Turing Machine lines up exactly with our notion of what is intuitively computable.

What is the difference between *deciding* and *recognizing*? HIDE ANSWER

A TM decides a language if it always halts with a YES or NO answer. A TM recognizes a language if it always correctly answers YES for strings in the language, but it may or may not halt with a NO for non-members.

What are the language classes P and NP? HIDE ANSWER

P is the set of languages decidable in polynomial time on a deterministic TM; NP is the set of languages decidable in polynomial time on a nondeterministic TM.

Does P = NP?

Did Scott Aaronson really write “everyone who could appreciate a symphony would be Mozart”? How does he feel about writing that these days? HIDE ANSWER

He did indeed write that but he regrets writing it now.

What are some variations we can make to Turing Machines that possibly affect complexity (if not computability)? HIDE ANSWER

Randomized machines, probabilistic machines, quantum machines.

How does Grady Booch distinguish the work in material domains (physics, chemistry, biology, etc.) from that of computer science? HIDE ANSWER

Scientists in material domains observe the cosmos and reduce it to simple principles; computer scientists start with simple principles and from them create new worlds bound only by the imagination.

Language Theory

What is an alphabet, in language theory? HIDE ANSWER

A set of symbols.

What is a string, in language theory? HIDE ANSWER

A finite sequence of symbols over a given alphabet.

If $w = aaabaaca$, what is $|w|$? HIDE ANSWER

8

How can we concisely and formally describe the language over the alphabet $\{a, b\}$ whose utterances contain all and only those strings made up of several a's followed by the same number of b's? HIDE ANSWER

$\{a^n b^n \mid n \geq 0\}$

How do we denote the concatenation of two languages, L_1 and L_2? HIDE ANSWER

L_1L_2

What is the difference between $\varnothing$ and $\{\varepsilon\}$? HIDE ANSWER

The former is the language of zero strings; the latter is a language with one string, namely the empty string.

For language L, what are $L\varnothing$ and $L\{\varepsilon\}$? HIDE ANSWER

$\varnothing$ and L.

What are variables in a grammar? HIDE ANSWER

Symbols outside the alphabet that are get replaced when generating (deriving) strings.

In what sense does a grammar define a language? HIDE ANSWER

The set of all strings derivable by the rules of the grammar is the language defined by the grammar.

How do we denote the language represented by the grammar G? HIDE ANSWER

$L(G)$

What does it mean for a grammar to be ambiguous? HIDE ANSWER

At least one string in the language defined by the grammar has more than one derivation tree.

What language does the grammar `s → s* | "(" s ")" | "[" s "]" | "{" s "}"` define? HIDE ANSWER

The language of properly nested and balanced brackets.

What is a context-free grammar? HIDE ANSWER

A grammar in which the left hand side of every rule is a single variable.

What is a right-linear grammar?

What is a type-1 grammar?

What is the difference between a generative and an analytic grammar?

Arrange R, FINITE, RE, DCFL, CFL, and REG in subset order.

What is the language class BPP? HIDE ANSWER

The set of languages that can be decided a probabilistic TM in polynomial time such that the probability of its answer being correct is $\geq 2/3$.

Arrange NP, EXPSPACE, EXPTIME, PSPACE in subset order.

What is $\boldsymbol{RE} \cap \text{co-}\boldsymbol{RE}$? HIDE ANSWER

$\boldsymbol{R}$ (the set of recursive languages)

A grammar is a special case of, but far more useful than, a _____. HIDE ANSWER

String Rewriting System

Syntax

What is syntax and how does it differ from semantics? HIDE ANSWER

Syntax is a specification of the structure of a language, while semantics specifies its meaning.

What are some spectra of syntactic forms in common programming languages? HIDE ANSWER

Pictures vs. text

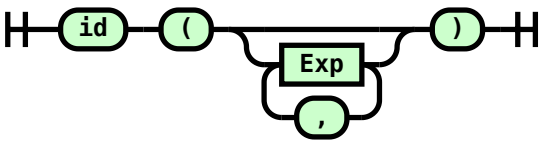
Symbols vs. words

Whitespace significance vs. insignificance

indentation vs. curly braces vs. ends vs. parentheses

prefix vs. infix vs. postfix

What does a syntax diagram for a function call look like, assuming the function being called must be a simple identifier? HIDE ANSWER



What are entities and identifiers? HIDE ANSWER

An entity is something that is manipulated by a program, such as a variable, constant, function, or literal. An identifier is a name you can bind to an entity.

What is the difference between the lexical and phrase syntax? HIDE ANSWER

The lexical syntax defines how individual characters are grouped into words, called tokens (such as identifiers, numerals, and punctuation); the phrase syntax groups words into higher level constructs (such as expressions and statements). These words can be separated by whitespace.

Why do grammars make a distinction between lexical and phrase syntaxes? Give two reasons. HIDE ANSWER

1. To avoid littering nearly every rule with whitespace sequences.
2. To conceptually simplify the syntax, breaking it down into two manageable parts.

How do we distinguish between lexical and phrase variables (in the grammar notation used in this class, and in Ohm)? HIDE ANSWER

Lexical variables begin with a lowercase letter; phrase categories begin with an uppercase letter.

What does it mean for a grammar to be **ambiguous**? Answer in precise, technical terms. HIDE ANSWER

A grammar is ambiguous if there exists a string that has two distinct derivation trees in the grammar.

How is operator precedence captured in a grammar? Give an example. HIDE ANSWER

We use multiple syntactic categories for expressions. The classic example, which gives addition lower precedence than multiplication, is:

```
Exp  → Exp addop Term
     | Term
Term → Term mulop Factor
     | Factor
```

What kinds of rules about program legality cannot be captured in a Ohm or context-free grammar? Find as many as you can. HIDE ANSWER

- Type checking
- Redeclaration of identifiers within a scope
- Use of an undeclared identifier
- Matching arguments to parameters in a call
- Access checks (public, private, etc.)
- Identifiers must be used in all paths through their scope
- A return must appear in all paths through a function
- Pattern match exhaustiveness
- In a subclass, abstract methods must be implemented or declared abstract
- All private functions must be called within a module

What kinds of rules about program legality can in principle be captured in a context-free syntax, but are not because the the required complexity of the rules? HIDE ANSWER

- Requiring `break` and `continue` in a loop (as this would require difference classes of statements)
- Requiring `return` in a function body (as this would require difference classes of statements)

In principle, can you capture the notion at all integer literals (in a hypothetical language) must be between -2147483648 and 2147483647? If so, why don't we? HIDE ANSWER

Yes, you certainly can, but the resulting specification is ugly af and really hard to decipher and understand.

What exactly is a **token**? HIDE ANSWER

A primitive element for a grammar's phrase structure, e.g., a keyword, identifier, numeric or simple string literal, or symbol.

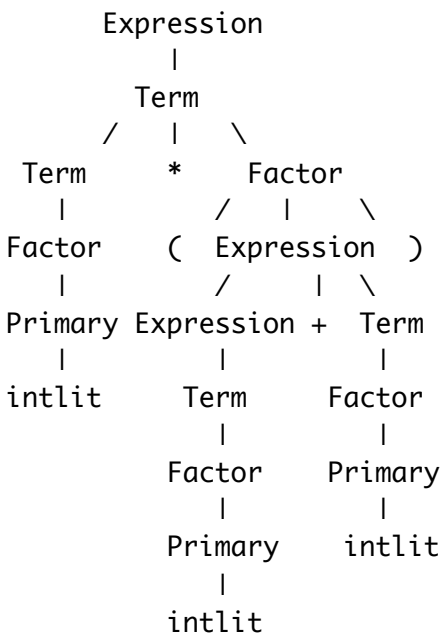
What happens during **tokenization**? What kinds of language constructs are "eliminated" in this process? HIDE ANSWER

Characters in the source code are grouped into tokens and comments and spaces are dropped. (Of course spaces within string literals are kept of course).

Given the expression `8 * (13  + 5)`, how many source code characters are there? When tokenized, how many tokens are there? How many nodes are there in the abstract syntax tree? HIDE ANSWER

14 characters, 7 tokens, 5 nodes.

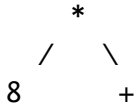
Draw the concrete syntax tree for the expression `8 * (13 + 5)`, assuming a grammar with categories named Expression, Term, Factor, Primary, and intlit. HIDE ANSWER



What is a concrete syntax? HIDE ANSWER

A precise specification of which strings (sequences of characters) are structurally legal programs.

Draw the abstract syntax tree for the expression `8 * (13 + 5)`. HIDE ANSWER



```
  /  \
13    5
```

Why do most language definitions provide a formal syntax but not a formal semantics? HIDE ANSWER

Syntax is normally context free and very easy to mathematically formalize; semantic definitions often feature quite a few ad-hoc constraints and lots of contextual information which is more clunky to formalize.

What is the difference between concrete syntax and abstract syntax?

What kinds of constructs appear in a concrete syntax but not in an abstract syntax? See how many items you can find. HIDE ANSWER

- Punctuation and delimiters
- Intermediate expression categories, e.g. Exp1, Exp2, Term, Factor, ...
- INDENT and DEDENT tokens

Give a simple rule that illustrates the difference between the `/` operator of a PEG and the `|` operator of a context-free grammar. (Hint: show a case in which the language defined by two "look-alike" grammars, one PEG, one CFG, are different. HIDE ANSWER

In a PEG, `A ← "a" / "ab"` only matches `"a"`, while the similar CFG defines the language {a, ab}.

Ohm

Can an Ohm grammar ever be ambiguous? HIDE ANSWER

Augh, yes and no, it depends. Certainly PEGs can't be ambiguous due to prioritized choice over non-deterministic choice and their prohibition against left recursion; but Ohm allows left-recursion so you can write:

```
A = A "+" A  --plus
  | "a"
```

which looks ambiguous. Ohm actually makes the operator here be right associative, so given the fact that Ohm is an implementation and always parses strings *exactly one way*, then no, its grammars are not ambiguous; however, this isn't really specified anywhere so maybe yeah, that grammar in some sense *is* ambiguous. See issues 55 and 56 for more information.

Given categories E (for expression) and T (for term), give an Ohm grammar rule to make the operator • on terms be **left-associative**. HIDE ANSWER

```
E = E • T  --binary
  | T
```

Given categories E (for expression) and T (for term), give an Ohm grammar rule to make the operator • on terms be **right-associative**. HIDE ANSWER

```
E = T • E  --binary
  | T
```

Given categories E (for expression) and T (for term), give an Ohm grammar rule to make the operator • on terms be **non-associative**. HIDE ANSWER

```
E = T • T
```

Here is an attempt to remove the need for operator precedence levels in a language design. Does it work? Why or why not?

```
E = E binaryop "(" E ")"
  | num
```

HIDE ANSWER

It does, but I'll admit it's hard to prove. Can someone help?

The parser generator in the Ohm system is unlike most others, in that it is not based on context-free grammars. What theoretical language description mechanism does it use? HIDE ANSWER

Parsing Expression Grammars, or PEGs.

Why do many languages make relational operators non-associative? HIDE ANSWER

Reasonable people can disagree on the meaning of `a<b<c`. Some people think it should be automatically expanded to `a<b && b<c`

How can we make the negation operator and the exponentiation operator not associate with each other? Show a grammar fragment. HIDE ANSWER

Put them on the same "level":

```
Exp7 = "-" Exp8
     | Exp8 "**" Exp7
     | Exp8
```

Why do language designers put functions like `sqrt` into a standard library (as opposed to being wired into the language, or left to a "third party" library? HIDE ANSWER

It is so commonly used that most people want or expect it without having to import an external library, and yet, it is not common enough to warrant its own operator wired into the core syntax of a language.

How does the Ohm rule `A = B | C` differ from the context-free grammar rule $A \rightarrow B \mid C$? HIDE ANSWER

In Ohm, the choices are ordered: when parsing, C is only "tried" if B does not match. When parsing a CFG, both alternatives may be tried.

What do the operators `?`, `*`, and `+` mean in Ohm? HIDE ANSWER

Optional, zero or more, one or more. (Same as in regular expressions, which is a good thing.)

What is wrong with the Ohm rule `WhileStmt = "while" Exp Block`? How do we fix this problem? HIDE ANSWER

If the expression begins with a letter, Ohm would still match the while statement even if there were no spaces between the word "while" and the expression! To fix this, create a lexical category `while = "while" ~idrest` and redefine the while statement rule as `WhileStmt = while Exp Block`.

How do we write a rule for JavaScript-style one-line comments in Ohm? HIDE ANSWER

```
"//" (~"\n" any)* "\n"
```

The Ohm notation

```
Factor = "-" Primary  -- negation
       | Primary
```

is actually an abbreviation for two separate rules. Give those two rules. HIDE ANSWER

```
Factor = Factor_negation
       | Primary
Factor_negation = "-" Primary
```

The Ohm rule `Exp = Term ("+" Term)*` fails to capture what aspect of the `+` operator in the syntax?

HIDE ANSWER

Associativity.

In Ohm, the construct `A ~B` matches an `A` that is not followed by a `B`. How do we match an `A` that *is* followed by a `B` (without consuming the B)? HIDE ANSWER

```
&(A B) A
```

In Ohm, if `g` is a grammar and `s` is a string, what does `g.match(s)` return? HIDE ANSWER

A match object.

Language Design

Before designing a programming language, what are some important important questions to answer?

HIDE ANSWER

- Why is the language needed (its purpose)?
- What is the language *for* (its scope)?
- Who is the language *for* (its audience)?

Compiler Basics

Describe the difference between an interpreter and a compiler. HIDE ANSWER

An interpreter runs a program; a compiler translates a program into a program in a different language.

Describe the difference between assemblers, compilers, and transpilers. HIDE ANSWER

An assembler translates assembly language to machine language; a compiler translates a anything into anything, but usually a high-level language into something lower level; a transpiler translates one high-level language into another high-level language.

Describe the difference between AOT and JIT compilers. HIDE ANSWER

An AOT (ahead-of-time) compiler translates the entire source program before running it. A JIT (just-in-time) compiler compiles as the code runs (as needed).

What does the **front-end** of a compiler do? HIDE ANSWER

Turn the source code into a conceptual, or intermediate, representation.

What does the **back-end** of a compiler do? HIDE ANSWER

Turn the intermediate representation of a program into target code.

What does the **middle-end** of a compiler do? HIDE ANSWER

Optimize, or improve, the intermediate representation.

What are the three forms of analysis of the compiler front-end? HIDE ANSWER

1. Lexical Analysis (tokenization)
2. Syntax Analysis (parsing)
3. Contextual Analysis (static, semantic)

Why are compilers split into a front end and a back end? Give the two most important reasons. HIDE ANSWER

1. You can't help but think of translation without an intermediate conceptual representation. 2. Your front-ends are reusable for many targets, and your backends are reusable for many source languages.

Critique the claim "Writing a transpiler means your compiler needs only a front end and not a backend." There's a kernel of truth to it, but it might not be wholly accurate. HIDE ANSWER

You *do* need a backend to generate the target code from an intermediate representation. Some purists might claim that to be a "real backend" you have to generate some really great, optimized assembly language for a serious target machine, so the claim all hinges on what is meant by "backend."

Is the program `tsc` a compiler or a transpiler? Do we care? HIDE ANSWER

Well, it does stand for "TypeScript *compiler*" so sure, it's a compiler because it compiles TypeScript into JavaScript. But then again, JavaScript is a high level language so you can call it a transpiler too. Both terms work. Don't be picky. Don't be that person. It's not worth getting worked up about.

What is LLVM? HIDE ANSWER

A set of tools to help you write compilers, perhaps the most visible feature of which is its intermediate representation to which many popular programming languages have been targeted to, and for which many code generators for popular computer architectures have been produced.

What is the JVM? HIDE ANSWER

A virtual machine that runs Java bytecode. Java bytecode is a hugely popular; compilers for dozens of different languages have been written to produce it.

What is a self-hosting compiler? HIDE ANSWER

A compiler written in the same language it compiles (i.e., the host language is the same as the source language).

What is a cross-compiler? HIDE ANSWER

A language in which the host and target languages are different (i.e., it runs on one machine but generates code for a different machine).

Give two reasons why compilers are often written in the language they compile. HIDE ANSWER

1. It helps in the process of porting a compiler to a new architecture. 2. If the compiler is an optimizing compiler, it can optimize...itself!

In a typical compiler, what does a parser produce? HIDE ANSWER

An abstract syntax tree.

Give stack machine style code for `x = y * (2 + z).` HIDE ANSWER

```
LOAD y
LOAD 2
LOAD z
ADD
MUL
STORE x
```

Rather than looking at a compiler as a monolithic translator, we should think of building up language processing libraries, or APIs. What advantages are gained from such a modular architecture? HIDE ANSWER

We can build syntax highlighters, linters, profilers, and debuggers directly in an IDE; we can embed the compiler in other applications.

Why must the source code for an indentation-sensitive language (such as Python) be pre-processed before being parsed by Ohm? HIDE ANSWER

The legal indentation amounts for a line depends on context, and Ohm cannot track this. Neither can any tradition CFG-based parser, either.)

What exactly is Esprima? HIDE ANSWER

A parser for JavaScript, which you can use form your own JavaScript programs.

In a language with string interpolation, what do we usually call the literal portions of a string? What do we call the interpolated portions? HIDE ANSWER

Quasis.

What is the difference between an expression and a statement? HIDE ANSWER

Expressions produce values; statements don't. Statements are executed only for their effect.

In many languages, expressions can appear within statements, but not the other way around. In JavaScript, however, statements can appear within expressions. Give an example. HIDE ANSWER

```
console.log(() => {if (true) return;})
```

In JavaScript, the left hand side of an assignment is not “just a variable“. What do we call that construct?

HIDE ANSWER

A pattern.

Static Analysis

When representing the built-in functions in a compiler, why don’t we create tree nodes for the function bodies?

HIDE ANSWER

The bodies of built-ins might not even be describable in the language itself (e.g., printing, writing to files, making network connections, and so on). It is left to the backend to either implement these or link to libraries that will be available at link time or run time.

Why do we need context objects in semantic analysis? HIDE ANSWER

By definition, semantic analysis is phase of compilation that is not context-free and thereby requires context! Context is loosely defined as what you’ve seen before, in some other parts of the AST, that you need right now.

Context objects in the Tiger compiler we looked at in class held (1) the local identifiers declared in the current scope, (2) whether or not we are inside a loop, (3) the current function we are in, and (4) the parent context. In a language like Java or Python, what else would be needed? Why? HIDE ANSWER

The current class, to deal with `this`. Also, the current set of imported packages, to resolve uses of imported entities.

Distinguish “static languages” and “dynamic languages” from the point of view of compiler writers.

HIDE ANSWER

Static languages are those with tons of legality rules (type checking, matching, counting, exhaustiveness, etc.) that have to be checked at compile time, so the compiler writer has to do a ton of work. Dynamic languages leave little for the compiler to do, as all those checks are pushed to run time.

What is meant by the **scope** of a binding? HIDE ANSWER

The region of the program text in which a binding is in effect. We care about this when writing a compiler because we have to find out which binding is active when we encounter uses of identifiers.

How would one allow for overloading of functions or methods in a semantic context object? HIDE ANSWER

Instead of context mapping an identifier to a single entity; it has to map each identifier into a list of entities.

For languages that scope local variables to the entire block in which they are defined, how is the temporal dead zone detected? HIDE ANSWER

It is the region of the block before the declaration.

Type checking is often concerned less with whether two types are identical, but rather when elements of one type T1 can be assigned to an Lvalue constrained to be of type T2. In what situations is this check made? HIDE ANSWER

(1) variable initialization, (2) assignment, (3) passing arguments to parameters, (4) returning from a function.

In the conditional expression `x ? y : z` of a typical statically-typed language, what type checking and inference rules would a compiler be required to enforce? HIDE ANSWER

Checks: the type of `x` must be boolean and the types of `y` and `z` must be compatible. Inference: the type of the entire expression is the least general type of both `y` and `z`.

How does a semantic analyzer check the legality of mutually recursive functions? HIDE ANSWER

In each block it first analyzes the signatures of each block and adds the function name and signature to the block's context. Then it analyzes the function bodies (on a second "pass" through the block).

What is the difference between a lexical and a syntax error? HIDE ANSWER

Lexical errors occur when a character sequence of the source program is such that a token cannot be formed. Syntax errors occur when tokens are in an arrangement that cannot be derived from the phrase-structure rules of the grammar.

What is the difference between a syntax error and a static semantic error? HIDE ANSWER

A syntax occurs when the program (after tokenization) cannot be derived by the grammar. A static semantic error occurs when the program is correctly matched by the grammar, but the compiler is able to deduce a violation of a legality rule, such as a type mismatch.

Why is “division by zero” not considered a dynamic semantic error in Java? HIDE ANSWER

An exception is thrown in this case and can be caught, with the program proceeding normally. Throwing and catching exceptions is well-defined and certainly does not violate any language rules.

In Java, the grammar allows `x < y < z`. So what exactly happens when the compiler encounters this code fragment (assuming all variables are in scope)? HIDE ANSWER

The operator is left-associative so `x < y` is checked. That is either a type error or a perfectly valid comparison assigned the type `boolean`. Booleans can’t be compared anyway, so the entire expression is *always* a type error, detectable at compile time! But it is NOT a syntax error.

Automata Theory

What kind of automaton accepts exactly the Regular Languages? HIDE ANSWER

Finite Automata (FA)

What kind of automaton accepts exactly the Context Free Languages? HIDE ANSWER

(Nondeterministic) Pushdown Automata (NPDA)

What kind of automaton accepts exactly the Type-1 Languages? HIDE ANSWER

Linear Bounded Automata (LBA)

What kind of automaton accepts exactly the Recursively Enumerable Languages? HIDE ANSWER

Turing Machines (TMs)

Turing Machines

Register Machines

Computability Theory

Regular Expressions

What are the metacharacters of modern regular expression languages? HIDE ANSWER

```
^   $   .   ?   *   +   |   (   )   [   ]   {   }   \
```

What exactly is it about metacharacters in regular expressions that make them special? HIDE ANSWER

The do not "stand for themselves"; instead, they enable *functionality* such as grouping, quantification, alternation, etc.

What is the regex that matches all and only those strings that begin with a hexadecimal digit, end with the same digit its starts with, and has a total length of between 3 and 8 characters? HIDE ANSWER

```
^([A-Fa-f\d]).{1,6}\1$
```

What is the regex that matches strings ending with 25 straight occurrences of Basic Latin small letter e? HIDE ANSWER

```
e{25}$
```

What is a **character class** in a regular expression? How many characters does it match?

Explain how the characters `]`, `^`, and `-` are interpreted in a regex character class.

How does the `s` flag affect a regular expression? HIDE ANSWER

When `s` is on, the dot matches the newline character.

How does the `m` flag affect a regular expression? HIDE ANSWER

When `m` is on, the `^` and `$` matches not only the start and end of a string, but the start and end of each line as well.

In order to get `\p{L}` working in JavaScript, what flag do you need? HIDE ANSWER

The `u` flag. This is needed because Unicode support for regular expressions was added late in JavaScript's life, and had a new flag not been added to enable this interpretation, backward compatibility would have been lost.

Write a regex to match a sequence of one or more characters, where the first is a letter (use `\p{L}`) or a `$`, and the following characters are letters, numbers, dollar signs, or periods, but in which you cannot have two periods in a row.

Recall that the metacharacters `^` and `$` stand for either the beginning and end of a line, OR the beginning and ending of the whole string, depending on a certain flag setting. How can we match the beginning and ending of a string, regardless of that flag setting? HIDE ANSWER

`\A` and `\Z`.

Why do you often see the characters `?:` at the beginning of a parenthesized expression in a regex?
HIDE ANSWER

Parentheses are sometimes needed for grouping but you don't need the group to be captured (and capturing is expensive).

How do you write negative lookahead, positive lookbehind, and negative lookbehind expressions in a regex?
HIDE ANSWER

- Negative lookahead: `(?!)`
- Positive lookbehind: `(?<=)`
- Negative lookbehind: `(?<!)`

What does the JavaScript expression `s.replace(/social(?=\s+distancing)/ig, "physical")` produce?
HIDE ANSWER

A string like `s` with all occurrences of the phrase `social distancing` —in any case, with any amount of whitespace between the two words—replaced with the phrase `physical distancing` with the same whitespace between the two words.

Parsing Theory

What are some advantages of hand-crafted parsers over parsers generated by a machine? HIDE ANSWER

Hand-crafted parsers and be faster and give much more precise error messages.

What are some applications and libraries that can generate parsers? HIDE ANSWER

YACC, Bison, ANTLR, Ohm

Intermediate Representations and Virtual Machines

Code Generation

Complexity Theory

Optimization

Can loop unrolling ever be unsafe? Why or why not?

Problems

The following problems generally require some research and a bit of time to work out solutions.

> *Some of the problems may refer to languages you have never heard of! If so, you can try solving the same problem with a language you are familiar with, or, better, look up the basics of the unfamiliar language so that you can take your best shot at the problem.*

Languages

These problems are courtesy of Phil Dorin.

1. Let $L = \{w \in \{0,1\}^* \mid w = w^R\}$.
   a. Is $\varepsilon \in L$?
   b. Is $101 \in L$?
   c. Is $101 \in L^2$?
   d. Is $1010 \in L^2$?
   e. Is $01101101110 \in L^*$?

2. Let $L_1 = \{w \in \{0,1\}^* \mid w \text{ has an even number of 0s and an odd number of 1s}\}$ and $L_2 = \{w \in \{0,1\}^* \mid w = w^R\}$.
   a. Is $\varepsilon \in L_1 L_2$?
   b. Is $10010110 \in L_1 L_2$?

c. Is $0010110101111 \in (L_1L_2)^*$?

d. Is $L_1 \subseteq L_1L_2$?

e. Is $L_2 \subseteq L_1L_2$?

f. Is L_1 countably infinite? If so, prove via an appropriate bijection; if not, prove via a proof to the contrary.

g. Is $L_1{}^*$ countably infinite? Prove or disprove.

3. Let L be the language denoted by the regular expression $0^*1 + 11(1 + 010)^*10$.

a. Is $\varepsilon \in L$?

b. Is $01 \in L$?

c. Is $0001 \in L$?

d. Is $0111 \in L$?

e. Is $10 \in L$?

f. Is $110100101111 \in L$?

g. Is $1101001011110 \in L$?

h. Is $111011101110 \in L^*$?

i. Is $\varepsilon \in L^*$?

j. Is L countable?

Grammars

1. Give grammars for the following languages, all over $\{0, 1\}$:

a. Strings of length ≥ 2

b. Odd binary numerals

c. Even binary numerals

d. Binary numerals divisible by 3

e. Binary numerals divisible by 4

f. Signed binary numerals that are negative (in 2's complement form)

2. Give grammars for the following languages, all over $\{a, b\}$:

a. Strings of length ≥ 2

b. Strings containing only a's, except that the first character could be a b

c. Strings containing at least 5 a's

d. Strings containing at least 5 consecutive a's

e. Strings not containing two consecutive b's

f. Strings whose 8th symbol from the right is a b

g. Strings having twice as many a's as b's

h. Strings having 3 times as many a's as b's

i. Palindromes of even length

j. Palindromes of odd length

k. Palindromes of any length (ww^R)

l. Strings whose first and last haves are the same (ww)

m. Strings with an odd number of a's and an even number of b's

n. $a^n b^n$

3. Give grammars for the following languages:

a. $\{a^i b^j c^i \mid j = 2i\}$

b. $\{a^i b^j c^i \mid j \leq i\}$

c. $\{a^i b^j c^i \mid j \geq i\}$

d. $\{a^i b^j c^i d^k \mid i, j, k \geq 1 \wedge k \text{ is a multiple of } 3\}$

e. $\{a^i b^j c^k \mid i \neq j \vee j \neq k\}$

f. $\{a^n b^n c^n \mid n \geq 0\}$

g. $\{a^n \mid n \text{ is a power of } 2\}$

h. $\{a^n \mid n \text{ is prime}\}$

i. $\{a^n \mid n \text{ is not prime}\}$

j. $\{w \in \{a, b, c\} \mid \#_a(w) = \#_b(w) = \#_c(w)\}$ HIDE ANSWER

We need the variables here because left hand sides of rules must always have at least one variable.

```
s   = (x y z)*    -- repeat xyz 0 or more times
x y = y x         -- switch them up all possible ways
x z = z x
y x = x y
y z = z y
z x = x z
z y = y z
x   = "a"         -- erase the variables
y   = "b"
z   = "c"
```

k. $\{a^i b^j c^i d^j \mid i, j \geq 0\}$ HIDE ANSWER

```
s     = (l r)?        -- start with left part and right part
l     = "a" l? x       -- generate a's on the left, counting them with x's
r     = "b" r "d" | y  -- generate equal nums of b's and d's leaving one y in the middle
x "b" = "b" x          -- move the x's to the right, in order to generate c's
x y   = y "c"          -- when the x hits the y, make a c for it
y     = ε              -- erase the y to finish it off
```

l. $\{a^i b^j c^k \mid 1 \leq i \leq j \leq k\}$ HIDE ANSWER

```
s     = "a" x? "b" z "c"  -- initial set up
x     = "a" x "b" y       -- if you generate an a, you must do b and c also
      | x? "b"? y         -- generate bc or c (keeping things increasing)
y "b" = "b" y             -- move y's to the right
y z   = z "c"             -- when y hits the z, make a c
z     = ε                 -- when the z is no longer needed, drop it
```

Turing Machines

1. Give a Turing Machine for multiplying a binary number by 8.

2. Give a Turing Machine for floor-dividing a binary number by 4.

3. Give a Turing Machine for negating a signed binary number (i.e., producing its two's complement).

4. Give a Turing Machine for incrementing a binary number.

5. Give a Turing machine that produces the string "1" if its input consists of all zeros, or the string "0" otherwise.

6. Give a Turing machine that determines whether a signed binary number is negative, i.e., that recognizes $\{w \in \{0,1\}^* \mid w \text{ is a negative signed binary number}\}$.

7. Give a Turing machine that determines whether a binary number is divisible by 5, i.e., that recognizes $\{w \in \{0,1\}^* \mid w \bmod 5 = 0\}$.

8. Give a Turing Machine ($\Sigma = \{A \ldots Z\}$) that erases its entire input and writes the message `HELLO`.

9. Give a Turing Machine ($\Sigma = \{a, b, c\}$) that appends its input to itself. For example, if your input was $abbca$ then the output would be $abbcaabbca$.

10. (Submitted by Amanda Marques) Give a Turing machine ($\Sigma = \{0, 1\}$) that determines if its input is a palindrome, i.e., that recognizes $\{w \in \{0,1\}^* \mid w = w^R\}$.

11. (Submitted by Amanda Marques) Give a Turing Machine ($\Sigma = \{a, b, c\}$) that appends the reversal of its input to itself, thereby generating a palindrome. For example, if your input was $abbca$ then the output would be $abbcaacbba$.

12. Give a Turing machine ($\Sigma = \{1\}$) that determines whether its input is exactly 8 symbols long, i.e., that recognizes $\{w \in \{1\}^* \mid |w| = 8\}$.

13. Give a Turing machine ($\Sigma = \{1\}$) that determines whether its input is exactly 88 symbols long, i.e., that recognizes $\{w \in \{1\}^* \mid |w| = 88\}$.

14. Give a Turing machine ($\Sigma = \{1\}$) that determines whether the length of its input is a power of 2.

15. (Submitted by Amanda Marques) Give a Turing machine ($\Sigma = \{0, 1\}$) that determines if its input *does not contain the substring 000*.

16. Give a Turing machine ($\Sigma = \{a, b\}$) that determines if its input has the same number of occurrences of a's as b's, i.e., that recognizes $\{w \in \{a,b\}^* \mid \#_a(w) = \#_b(w)\}$.

17. Give a Turing machine that recognizes $\{a^n b^n \mid n \geq 0\}$.

18. Give a Turing machine that recognizes $\{a^n b^n c^n \mid n \geq 1\}$.

19. Give a Turing machine ($\Sigma = \{0, 1\}$) that determines if its input contains at least three zeros (not necessarily contiguous).

20. Give a Turing machine ($\Sigma = \{a, b\}$) that determines if its input contains an even number of b's.

    HIDE ANSWER

```
EVEN,a,a,R,EVEN
EVEN,b,b,R,ODD
ODD,a,a,R,ODD
ODD,b,b,R,EVEN
EVEN,#,#,L,ACCEPT
```

21. Give a Turing machine that determines for two strings, whether the first is longer than the second, given the following set up. The input alphabet is $\Sigma = \{a, b, \bullet\}$ and the input will be in the form $w \bullet x$ for strings w and x. Your TM should recognize $\{w \bullet x \mid w, x \in \{0, 1\} \wedge |w| > |x|\}$.

22. Give a Turing machine that determines for two strings, whether the first is a substring of the second, given the following set up. The input alphabet is $\Sigma = \{a, b, \bullet\}$ and the input will be in the form $w \bullet x$ for strings w and x. Your TM should recognize $\{w \bullet x \mid w, x \in \{0, 1\} \wedge w \textbf{ is a substring of } x\}$.

23. Give a Turing machine that computes the sum of two unary numbers, given the following set up. The input alphabet is $\Sigma = \{1, \bullet\}$ and the input will be in the form $w \bullet x$ for strings w and x. Your TM should output 1^{i+j} when it sees the input $1^i \bullet 1^j$.

Compilation in Practice

1. Find, and link to, real-life examples of self-hosting and cross compilers.

2. Suppose a new computer called the X1234 has just come out and it doesn't have a Swift compiler. But you want to make a resident Swift compiler on that machine. Fortunately you have a resident Swift compiler that runs on a MIPS machine. Describe exactly how you can construct the desired resident Swift compiler for the X1234 using the one for the MIPS.

Programming

Don't worry if there are languages here you don't know. Do some research. Learn something new today.

1. In the notes on Theories of Computer Science we saw how to express the odd-number test in Lambda Calculus notation, Lisp, Python, JavaScript, Java, Ruby, Clojure, Kotlin, and Swift. Show how, in each of these notations or languages, to express a function to cube a number. (Research may be required, as some of these languages may be new to you.)

Syntax

Some of these problems refer to syntactic forms not covered in lecture, but should be answerable after studying my course notes on Syntax.

1. The following is a failed attempt to write a grammar for the language $L = \{w \in \{a, b\}* \mid w \text{ has exactly twice as many } a\text{s as } b\text{s}\}$:

   ```
   S → aab | aba | baa | aaSb | abSa | baSa | aSab | aSba | bSaa | SS
   ```

   a. Prove that ***aaabbbbaaaaa*** is not in the language generated by this grammar.

   b. Give a correct context free grammar for L (and don't forget, that the empty string belongs, too).

   HIDE ANSWER

   There's a little backstory to this problem. I was given this problem while a student in UCLA in early 1988. The TA gave the incorrect answer above. I showed the problem and the TA's solution to Phil Dorin, who didn't think the solution was right and worked on and off for 10–15 years to prove it wrong. Finally he wrote the following message to his teacher, Sheila Greibach:

   > Among the reasons that I have wanted to write is that, many years ago, my colleague, Ray Toal, whom you know, passed along a set of problem solutions that he received while studying at UCLA. They were for the 181 course—his instructor at the time was a fellow named Gabriel Robins, which probably tells you how long ago this was!—and they contained an error that I had always meant to report to you. (It's been so long now that it has probably been corrected, but I'll sleep a lot better once I've sent this off.) Specifically, the problem was to give a cfg that generated the set of all strings over alphabet $\{a, b\}$ with exactly twice as many ***a***s as ***b***s, which, I believe, was also a problem in an earlier edition of Hopcroft and Ullman. In any event, he gave the following solution, which he attributed to Lui:
   >
   > ```
   > S → SS
   > S → aaSb
   > S → abSa
   > S → baSa
   > S → aSab
   > S → aSba
   > S → bSaa
   > S → aab
   > S → aba
   > S → baa
   > ```
   >
   > Now, I am going feel awfully much like an idiot if I am wrong about this, but... how does this grammar produce the string ***aaabbbbaaaaa*** (that is, three ***a***s, followed by four ***b***s, followed by five ***a***s)? I have managed to prove to myself that it simply can NOT produce this string, and I wonder if I should trouble you to look at it and let me know. (Technically, the grammar is also missing a rule for producing the empty string, which is also in the language, but that's another matter.)
   >
   > I do believe that a correct grammar is:
   >
   > ```
   > S → [empty string]
   > S → SS
   > S → aSaSb
   > S → aSbSa
   > S → bSaSa
   > ```

> I've also worked the problem from the other direction: I constructed a npda, converted it to a cfg, and simplified it (by removing useless symbols, etc.)—but the resulting grammar doesn't look anything like the above ones, so this didn't provide much new insight.

Prof. Greibach gave a nice reply, and as part of it managed to state almost nonchalantly the "obvious" proof (at least to her—in a single sentence!) of non membership of ***aaabbbbaaaaa***:

> It does indeed fail on the example you gave since the first rule applied could not be any of those starting S → a... or S → b... and S → SS cannot be used because the example is not the concatenation of 2 words in the language.

2. We've seen the EBNF form $A\hat{}B$ which denotes $A \mid ABA \mid ABABA \mid \ldots$. Such a form makes it convenient to write rules involving *separators*, such as

```
IDLIST → ID ^ ","
```

This form can also be used to model a construct representing one or more As, rather than using AA^* or A^*A. Show how to do this.

HIDE ANSWER

Let ε represent the empty string. Then we can use $A\hat{}\varepsilon$.

3. Here are a few Ohm grammar rules from the Ada programming language:

```
Exp     = Exp1 ("and" Exp1)* | Exp1 ("or" Exp1)*
Exp1    = Exp2 (relop Exp2)?
Exp2    = "-"? Exp3 (addop Exp3)*
Exp3    = Exp4 (mulop Exp4)*
Exp4    = Exp5 ("**"  Exp5)? | "not" Exp5 | "abs" Exp5
comment = "--" ~"\n" any
```

   a. What can you say about the relative precedences of `and` and `or`?
   b. If possible, give an AST for the expression `X and Y or Z`. (Assume, of course, that an Exp5 can lead to identifiers and numbers, etc.) If this is not possible, prove that it is not possible.
   c. What are the associativities of the additive operators? The relational operators?
   d. Is the `not` operator right associative? Why or why not?
   e. Why do you think the negation operator was given a *lower* precedence than multiplication?
   f. Give an abstract syntax tree for the expression `-8 * 5`.
   g. Suppose the grammar were changed by dropping the negation from Exp2 and adding `- Exp5` to Exp4. Give the abstract syntax tree for the expression `-8 * 5` according to the new grammar.

4. The official grammar of the C programming language has over a dozen levels of operator precedence defined within the grammar. Write this subset of C syntax using Ohm.

5. Give grammars for the languages:
   a. $\{a^n b^n c^n \mid n \geq 0\}$

b. $\{a^i b^j c^k \mid i = j \text{ or } j = k\}$

c. $\{ww \mid w \in \{a, b\}*\}$

6. Describe each of the following languages in both EBNF and Ohm:
   a. $\{w \in \{a, b, c\}* \mid w \text{ has at most one occurrence of any symbol}\}$
   b. $\{a^m b^n c^{m+n} \mid m \geq 1 \wedge n \geq 1\}$
   c. Palindromes over $\{a, b\}$
   d. $\{a^m b^n \mid m \geq n\}$
   e. Strings of parentheses, brackets and braces, all properly balanced and nested
   f. Semicolon terminated statements
   g. Comma separated expressions
   h. Strings over $\{a, b, c, d, e\}$ containing at most one occurrence of any symbol

7. EBNF generally uses
   - $A\ B$ to mean exactly one A followed by exactly one B
   - $A?$ to mean zero or one A
   - $A*$ to mean zero or more As
   - $A \mid B$ to mean either exactly one A *or* exactly one B

   Suppose I wanted to add a new one:

   - $A_1 \# A_2 \# \ldots \# A_n$ to mean "a non-empty string in which each of the A_is appears zero or one times, but in any order."

   Show how to write $A \# B \# C$ using only the conventional EBNF markup.

   HIDE ANSWER

   $A \mid B \mid C \mid AB \mid AC \mid BA \mid BC \mid CA \mid CB \mid ABC \mid ACB \mid BAC \mid BCA \mid CAB \mid CBA$

8. Suppose we are designing a language and wish that no identifier could be exactly three characters long and end with `"oo"` (or `"o0"` or `"0o"` or `"00"`).
   a. Write a regex for alphanumeric strings beginning with a letter that are *not* three characters long ending case-insensitively with `"oo"`.
   b. Give a (lexical) Ohm rule to define identifiers as any string of alphanumerics and underscores, beginning with a letter, that satisfies our wish.

9. Write a function in the language of your choice that returns whether its input string is a three character alphanumeric string ending, case insensitively, in `"oo"`. Do this by matching against a regular expression.

10. Describe, in English, the languages expressed by these regular expressions:
    a. `[01]*(10111[01] | 11[01][01][01][01])[01]*`
    b. `([bc]*a[bc]*a[bc]*)*`

c. `0*1 | 0*10`

d. `c*a[ac]*b[abc]*`

11. Write regular expressions that:

a. Match octal constants in C HIDE ANSWER

0[0-7]*

b. Match hexadecimal numerals divisible by 8 (signed or unsigned!)

c. Match strings that begin with unsigned 32-bit hexadecimal numerals divisible by 16

d. Match entire strings that are sixteen-bit hexadecimal numerals (signed or unsigned!) divisible by 8

e. Match entire strings that are unsigned binary numbers, of any size, divisible by 8

f. Match floating point constants that are not allowed to have an empty fractional part and can have no more than three digits in the exponent part

g. Match floating point constants that are allowed to have an empty fractional part and can have no more than four digits in the exponent part

h. Match identifiers that are strings of letters, digits, and underscores, that begin with a letter, are not allowed to end with an underscore, and cannot contain two successive underscores anywhere in the text.

i. Match non-empty words consisting of the letters a-z whose first and second halves are the same (i.e., in set notation: $\{ww \mid w \in \{a..z\}+\}$)

j. Match entire character strings that contain neither the substring `"return"` nor `"retry"`

k. Match entire strings of that must be made up of lowercase Basic Latin letters only and that contain neither the substring `"exit"` nor `"exec"`

l. Match entire strings that contain neither the substring `"exit"` nor `"exec"`

m. Match all words in a string (use `\b` for word boundaries) that are preceded by the word “ `the` ”.

n. Match words containing two adjacent double-letters.

o. Match strings of digits not preceded by a dash. HIDE ANSWER

(?<!-|\d)\d

- We’ve seen that one way to deal with ugly code in curly brace languages is to require blocks in compound statements; for example:

```
IFSTMT → 'if' '(' EXP ')' BLOCK
         ('else' 'if' '(' EXP ')' BLOCK)*
         ('else' BLOCK)?
BLOCK → '{' STMT* '}'
```

What if we tried the same approach in a language with a syntax like Ruby (or Fortran or Modula — languages using a terminating end)? We might get a grammar like this:

```
IFSTMT → 'if' EXP 'then' STMT+
         ('else' 'if'  EXP 'then' STMT+)*
         ('else' STMT+)?
         'end'
```

Is this grammar left recursive? Is it $LL(k)$? Why or why not? Is this bad?

- Is this grammar an LL grammar?

```
A → B C
B → a | b?c?
C → c | BA
```

If this grammar is not LL, make one that is (that defines the same language of course). Give a set of syntax diagrams for the original diagram, and if another is needed, for the new grammar as well.

- Here's an Ohm grammar:

```
S = A M
M = S?
A = "a" E | "b" A A
E = ("a" B | "b" A)?
B = "b" E | "a" B B
```

1. Describe in English, the language of this grammar.
2. Draw a parse tree for the string `"abaa"`
3. Prove or disprove: "This grammar is $LL(1)$."
4. Prove or disprove: "This grammar is ambiguous."

- Here's a grammar that's trying to capture the usual expressions, terms, and factors, while considering assignment to be an expression.

```
Exp         ::=  id ':=' Exp | Term TermTail
TermTail    ::=  ('+' Term TermTail)?
Term        ::=  Factor FactorTail
FactorTail  ::=  ('*' Factor FactorTail)?
Factor      ::=  '(' Exp ')' | id
```

1. Prove that this grammar is not $LL(1)$. HIDE ANSWER

   Both alternatives for `Exp` expand to a string beginning with `id`.

2. Rewrite it so that it is $LL(1)$.
3. Rewrite the grammar as a PEG.
4. Write the grammar using Ohm, using left-recursion.

Language Features

1. C does not allow structures (i.e., non-atomic objects) to be tested for equality. Ada does. Maybe the designers of C wanted to keep things simple. How exactly would equality operations for structures complicate a C compiler or the runtime system?

2. If possible, write a program in Modula 3 that makes a variable point to itself. That is, for some designator `X`, make it so that `X^ = X`. If this is not possible, state why it is not possible.

3. If possible, write a program in Ada that makes a variable point to itself. That is, for some designator `X`, make it so that `X.all = X`. If this is not possible, state why it is not possible.

4. If possible, show how to make a ML variable `x` of type `x` such that `x.x = x`, or state why this is impossible.

5. If possible, show how to make a Hana variable `x` of type `x` such that `x.x == x`, or state why this is impossible.

6. In C++ you can say `(x += 7) *= z` but you can't say this in C. Explain the reason why, using precise, technical terminology. See if this same phenomenon holds for conditional expressions, too. What other languages behave like C++ in this respect?

7. Consider the `continue` statement of C.
   a. What kind of static semantic checks are required for this statement?
   b. Give an example piece C code that has a continue statement in it, and show the intermediate and target code for it.

8. Some languages do not require the parameters to a subprogram call to be evaluated in any particular order. Is it possible that different evaluation orders can lead to different arguments being passed? If so, give an example to illustrate this point, and if not, prove that no such event could occur.

9. Ada allows subprograms to be objects, as in the following code fragment:

```
type Real_To_Real is access function (Real) return Real;
type Foo is access procedure (Integer; in out Boolean);
Sine, Cosine: Real_To_Real;
P: Foo;
Q: Real_To_Real;
function Integrate (F: Real_To_Real; A, B: Real);
...
function Square (X: Real) return Real is
begin
    return X * X;
end;
...
Put (Integrate(Square'Access, 3, 10));
Q := Cosine;
if Q(Pi) > X then ...
```

   Describe the semantic rules relating to this facility in Ada, and how you would enforce them in a compiler.

10. It is a well-known irritation that Ada does not allow you to write array aggregates for zero- or one-element arrays, e.g., `A := (3)` gives a static semantic error when A is a one-element array of Integer. Why is this so? Propose a (trivial) syntactic extension to Ada that would remove this irritation.

11. In Ada, the declarations

```
X: Integer := X + 1;
Foo: Foo;
Bar: Real := Bar(Foo);
```

(where global declarations of `X`, `Foo` and `Bar` are visible) are all illegal, since a declaration of an identifier hides global declarations of the same name immediately at the point it appears in the text, but the identifier may not be used until its declaration is complete. Give an alternate interpretation under which these declarations would be legal and explain the advantages and disadvantages of it from both the programmer's and the compiler writer's perspectives.

12. In C++ it is not permitted to have two functions that differ only in return type overload each other. In Ada it is allowed. What is the reason for this situation? Even though Ada does allow this flexibility in overloading, the compiler needs some sophistication. What exactly is involved? Be very precise in your explanation and illustrate it with code fragments.

13. Some programming languages require that in order to have mutually recursive functions, the programmer first define the first function's signature (name, return types, parameters and parameter types), then the entire second function, then the entire first function. For example, in C++:

```
int f(int x, char y);
void g(int x) {if (x < 0) f(2, 'c');}
int f(int x, char y) {g(randomInteger());}
```

In C++, when `f` is finally declared, the names of the formal parameters don't have to be repeated exactly as they appeared in the incomplete specification. But in Ada they do. Explain why the Ada rule makes life much easier for the compiler writer.

14. Many languages have a syntax rule

```
DESIGNATOR → DESIGNATOR "." ID
```

for specifying variables made up from a record and a field of the record. But sometimes it can have the additional interpretation that the `DESIGNATOR` to the left of the dot was the name of a (visible) subprogram and the ID was an object declared immediately inside that subprogram. Show how to rearchitect the entity class hierarchy to support this.

15. An online troll suggested that JavaScript was really confusing because it uses square brackets for array expressions, instead of simple parentheses. "After all," this person says, "in English we don't use square brackets much if ever, so it should have had regular parentheses." Can we change JavaScript to work this way, and in doing so, affect *only* array

expressions, that is, not cause any ambiguities in existing JavaScript code not involving arrays? If so, show what the following would look like:

a. The assignment of a four-element array expression to a variable

b. The assignment of a one-element array expression to a variable

c. The assignment of a zero-element array expression to a variable.

and explain why your solutions satisfy the restriction that the change only affects expressions with array expressions.

16. Explain why including the return type of a function in the criteria for distinguishing functions for the purpose of overloading would greatly increase the complexity of a Hana compiler.

Abstract Syntax

1. Draw a JavaScript abstract syntax tree for the following script (you can use Esprima to check your work):

```
let [x, y] = Array.repeat(10|-2, 2);
function f({x, y}, ...p) {
  return q => `"Say ${y/p[0].x} today`;
}
```

2. Show a Java abstract syntax tree for:

```
static protected synchronized long g(Object... m) {
    for (int y : f(x)) {
        x = p.data[0] * (3<<   7|-  x---c);
    }
}
```

3. Draw the abstract syntax tree for the following C fragment:

```
for (int i = x-3; q<=4&m.z[r |- 4]&2-8*r>- 5/~x;) {
    while (a) {
        y;
        2,y;
    }
}
```

4. Draw the abstract syntax tree for this C function declaration:

```
void f(int x,...) {
    struct e {
        double x;
        struct e *c[10];
        char* (*f)();
    };
    struct e p;
```

```
    exit(p.c[1]->f()[6 |~ x+2 >> x]);
}
```

5. Draw the abstract syntax tree for this C function declaration:

```
int abc() {
      return x = 4&x---*&y.m[-9];
}
```

6. Give an abstract syntax tree for the following Java code fragment:

```
if (x > 2 || !String.matches(f(x))) {
    write(-3 * q);
} else if (! here || there) {
    do {
       while (close) tryHarder();
       x = x >>> 3 & 2 * x;
    } while (false);
    q[4].g(6) = person.list[2];
} else {
    throw up;
}
```

7. Draw the abstract syntax tree for the following Java compilation unit. (Make sure it is fairly abstract):

```
package p;
class C implements A {
    public static A x = new   t[3];
    Socket s () {
        while (x -  6>p  |    e || q +- p) {
            this.x[3] = !v+++t;
        }
    }
    {System.out.println("ooh");}
}
```

8. Draw the AST for the following C fragment:

```
(a = 3) >= m >= ! & 4 * ~ 6 || y %= 7 ^ 6 & p
```

Assembly and Machine Language

1. Write an assembly language program that displays a multiplication table of size 12 × 12.

2. Write in assembly language a translation of the following C function

```
double f(int x, double y) {
    return 4 * x + y;
}
```

3. Under what circumstances can you safely replace the x86 code fragment

```
        je    L6
        jmp   L4
L6:
```

   with the single instruction `jne L4`?

4. Show that the addressing modes immediate, absolute memory, and register indirect can be simulated by register and register-offset alone.

5. Show the target code that is generated for the source statement `X := Y;` where `X` and `Y` are both 32-bit integers that are one step down the static chain from the current subprogram, by a code generator which emits access code for the two values independently. Assume X is at offset -8 and Y is at offset -12. How many registers are used? Then generate code for this statement by hand, intelligently.

6. Suppose the variable A was declared in an Ada program with

```
type array (21..38) of String(1..10)
```

   and happened to have offset -42 in the frame of the subprogram in which it was declared. Suppose further that the variable `J` was declared in the same subprogram and had offset -26.

   a. Show the target code that loads the value of `A(J-1)` into register eax that would be generated naïvely. Do not forget to show the bounds checking!

   b. Show target code to load the value of `A(J-1)` into register eax in which the "-1" computation is folded in to the computation of the base address of A. Note that the bounds checking code will look a little different than in part (a).

7. Write an assembly language program that takes zero or more command line arguments, which should all be integers, and displays the average of the parameters to standard output.

8. Occasionally a compiler may output a sequence such as

```
        mov    [ebp-8], eax
        mov    eax, [ebp-8]
```

   The second instruction might be able to be removed. But whether we *are* able to remove this instruction is undecidable. Why, exactly?

9. The x86 has an `enter` instruction which automatically makes a display. Research this instruction. Suppose a Carlos program had the following structure (indentation determines nesting):

```
function f, parameters: [x,y], locals: [a]
    function g, parameters: [c], locals: [p,q,r,s]
        function h, parameters: [a], locals: []
    function k, parameters: [], locals: [z]
```

a. Show what the runtime stack looks like from the call sequence f→g→k→g→h→h→f→k

b. What does the generated assembly language look like when trying to access the value of f.x from h?

c. Which parts of the Carlos compiler need to be rewritten to use this instruction?

10. The ENTER instruction is rarely used because it is slow. Show how slow it is by doing the following. Prepare a table with four columns. The left column will be:

```
enter n, 0
enter n, 1
enter n, 2
enter n, 3
...
```

and so on. The second column will be the number of clock cycles required on a Pentium for the particular ENTER instruction. The third column will be code equivalent to the ENTER instruction. For example, ENTER n, 1 is equivalent to:

```
push ebp
mov  ebp, esp
push ebp
sub  esp, n
```

The fourth column will be the number of clocks for the code in column 3.

11. Show x86 code for the expression

```
x / y > (3 * x) || z || x < 3
```

where the "||" operator is short-circuit, and the variables x, y, and z are all integer variables. Put the value of the expression in eax. Write the best possible code you can for the Pentium 4 processor.

12. Write an assembly language function to compute $\frac{\sin(\log(x))}{y-7}$ where x and y are two double (64-bit float) parameters. Use the x86 C calling convention. Also write a C program that calls the function and displays the result.

13. Write an assembly language function to compute the log base a of b, where x and y are two double (64-bit float) parameters. Use the x86 C calling convention. Write a C program for the unit tester (with at least 10 assert statements).

14. Write an assembly language function to compute $\frac{y}{\sin\log\operatorname{atan2}(y,x)}$ where x and y are two double (64-bit float) parameters. Use the x86 C calling convention.

15. Write an x86 assembly language program that sets every third byte of the three megabyte section of memory starting at address ***b***. Use the MMX registers.

16. Write an assembly language function that returns the dot product of two single-precision floating point arrays using the XMM registers. Implement a unit tester in C.

17. Write an x86 assembly language function that returns the sum of the reciprocals of all the elements in an array of doubles. Use the C calling convention (so the function accepts the array and a length).

18. Write an assembly language version of the following, using an `LEA` instruction for the 3n+1 computation:

```
int C(int n) {
    int count = 0;
    while (n != 1) {
        n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
    }
    return count;
}
```

19. Show both naive and optimized intermediate code (entity graph), and both naive and optimized assembly language for:

```
if (x % 4096 == 0) {printf("Don't say \66;\6f;\6f;!");}
```

    Hint: you need strength reduction, too.

20. One kind of strength reduction is replacing division by a power of two with an arithmetic right shift, for example

```
sar eax, 10          to divide by 1024
sar eax, 8           to divide by 256
```

    This optimization is not safe. Explain why. Show how to make it safe, and explain both why your optimization works and why it is safe.

21. Write an x86 assembly language function that takes in four doubles and returns the product of the largest and the smallest argument. Assume the function will be called from a C program built under gcc running on a Pentium II or above. Note that you need to respect the calling convention. **Do not use conditional jumps in your code**.

22. Give highly optimized x86 code for the following:

```
for j := 5 to y do
    y := j * 7 + c;
    printInteger(y - 4);
end loop;
```

    where `y` and `c` are local variables in the current procedure at offsets -12 and +16 respectively. Remember that the range is evaluated only once, the whole loop is skipped on the empty range, etc.). Make sure you respect the overflow

semantics! Identify any induction expressions and explain how you optimized them. Compare your hand-written code with that generated by a real compiler.

23. Write an x86 assembly language function to return the product of its input (which must be a double) and 7.0, **without using multiplication or loops**. USE AT MOST 4 ADDITIONS. The return type is double. Assume the function will be called from a C program built under gcc.

24. Write the following in assembly language (use the C calling convention). It is supposed to compute $a*log_{10}(b)$. Use the `fyl2x` and `fldl2t` instructions.

```
double f(double a, double b);
```

25. What does this code do? For what ranges of n does it make sense?

```
mov eax, n
shl eax, 23
add eax, 3f800000h
mov [esp-4], eax
fld dword [esp-4]
```

26. Generate code for the following basic block:

```
y := x * 4 + z;
z := p * y;
y := z;
x := z / y << x;
```

Runtime Systems

1. A naïve way to implement a runtime system for a language with exceptions is to place two return addresses in an activation record. Sketch a small Ada or C++ function that can throw (a possibly user-defined) exception, and a code fragment that calls the function. Give astack frame layout with two return addresses, one is the normal return address and the other is the address of the handler in the caller. Show the assembly language for the caller and the function itself.

2. Discuss advantages and disadvantages of a subprogram call implementation in which (a) the calling subprogram saves all registers and (b) the called subprogram saves all registers. Explain why the x86's C calling convention is a nice compromise.

3. In a language that supports recursion, there may be multiple activations of a subprogram on the dynamic chain, and hence stack allocations of frames are generally used. However, subprograms that do not themselves make calls need not use stack frames. More generally, any subprogram that can never appear twice on a dynamic chain does not require a stack frame. Describe how to compute the set of all such subprograms at compile time.

4. What exactly must be the case for a subprogram to not need a static link in its stack frame? Think up as many cases as possible.

5. In Ada, C, and C++ arrays and records (structs) can be allocated on the stack, not just on the heap. When making assignments of aggregates to variables, compilers usually generate code to deposit the values in temporary storage. Why is this necessary in general? After all, in

```
Weekdays := Day_Set(False, True, True, True, True, True, False);
```

we could construct the aggregate directly in the variable `Weekdays`. Give an example of an assignment statement that illustrates the necessity of constructing an aggregate in temporary storage (before copying to the target variable).

Errors

1. Identify the following errors as syntactic, static semantic, or dynamic semantic (runtime): If no language is mentioned for a particular case, it probably does not matter. Assume either C or Ada and write your assumption.
   a. Redeclaration of an identifier.
   b. Unbalanced parentheses.
   c. Applying an operator to an element of the wrong type.
   d. Array index out of bounds (in C, in Ada, ...).
   e. Division by zero.
   f. Semicolon after a block in C.
   g. Wrong number of arguments supplied to a call.
   h. Assignment of a variable of type T to a variable of type subtype of T where the first variable is out of the range of the second in Ada.
   i. An unwanted infinite loop.
   j. Dereference of a null pointer.
   k. Application of the "." to an identifier which is not a field of the record.
   l. Use of an uninitialized variable.
   m. `for x (a) {printf("*"); x = x++;}` in Hana.

2. Which if the following expressions are legal in Java (assuming x and y are integer variables)? State why they are legal or why they are not.
   a. `x---y`
   b. `x-----y`

3. Classify the following as a syntax error, semantic error, or not a compile time error at all. In the case where code is given, assume all identifiers are properly declared and in scope. All items refer to the Java language.
   a. `x+++-y`
   b. `x---+y`
   c. incrementing a read-only variable

d. accessing a private field in another class

e. Using an uninitialized variable

f. Dereferencing a null reference

g. `null instanceof C`

h. `!!x`

4. Classify the following as (a) lexical error, (b) syntax error, (c) static semantic error, (d) dynamic semantic error, or (e) no error.

   a. A function call with no matching signature in Hana.

   b. A function call with no matching signature in C.

   c. `x < y < z` in Hana, where x and y are ints and z is a boolean.

   d. `x < y < z` in C, where x and y and z are all ints.

   e. `3[a]` in Hana, where `a` is an array variable.

   f. `3[a] in C`, where `a` is an array variable.

   g. `char x = '\a';` in Hana.

   h. `char x = '\a';` in C.

   i. Value returning function without a return statement, in Hana.

   j. Value returning function without a return statement, in C.

   k. Semicolon after a block, in Hana.

   l. Semicolon after a block, in C.

5. Classify each of the following, assuming a typical statically typed language, as a (a) lexical error, (b) syntax error, (c) static semantic error, (d) dynamic semantic error, or (e) no error.

   a. Invoking an array constructor that accepts a length and produces an empty array of that length, with a negative argument

   b. Semicolons instead of commas in identifier lists

   c. An identifier that is 33 characters long

   d. Applying a length operator or function to a read-only array variable

   e. Applying a length operator to a struct

   f. Applying the `sin` standard function to an integer

   g. The expression `x < y < z` where x and y are integers, and z is a boolean.

   h. Having the wrong number of arguments in a struct's constructor.

HIDE ANSWER

a. Dynamic semantic

b. Syntax

c. No error

d. No error

e. Static semantic

f. No error

g. Syntax

h. Static semantic

Theory

1. The reachability problem is to determine for a given instruction, whether or not it might be executed for some run of the program. To optimize a program for space, we need to solve the reachability problem and remove all unreachable instructions. Show that this is impossible by reducing the halting problem to the reachability problem.

Optimization

1. Give three examples of how aliasing can occur (you can use examples from several different languages). How does aliasing make copy propagation difficult? When, if ever, can an algorithm determine that a entity cannot possibly be aliased?

2. Optimize the following. Show your work (that is, show a few intermediate steps toward your final solution, recording the optimizations you performed. You can abbreviate CP=copy propagation, CF=constant folding, DCE=dead code elimination. You'll want to use more than just these three techniques.

```
L1:
    r0 := x
    z := 6
    r1 := 4 - r0
    r2 := 3 >= r1
    if r2 == 0 goto L2
    r3 := y + 4
    r4 := *r3
    z := r4
L2:
```

3. Write, by hand, a super-efficient Squid fragment for the following Hana fragment:

```
struct s {int x; int y; string s;}
s a = new s {
    codepoint(getChar()), codepoint(getChar()), getString()};
while (a.x++ < a.y) {print($a.s[1]);}
```

4. Here's some Hana code that prints the elements of an integer array separated by commas:

```
for (int i = 0; i < #a; i++) {
    print($a[i]);
    print(", ") if (i != #a-1);
}
```

With optimizations turned off, my compiler produces:

```
p0:
  copy 0, i1
L0:
  copy [i0-4], r0
  less i1, r0, r1
  jz r1, L1
  assert_not_null i0
  copy [i0-4], r2
  assert_in_range i1, 0, r2
  mul i1, 4, r3
  add i0, r3, r4
  copy [r4], r5
  to_string r5, r6
  param r6
  call __print, 4
  copy [i0-4], r7
  sub r7, 1, r8
  not_equal i1, r8, r9
  jz r9, L2
  param s0
  call __print, 4
L2:
  inc i1
  jump L0
L1:
  exit
s0:
  [44, 32]
```

a. Describe, in high-level terms, what each of the **assert** tuples are doing. Are both of them necessary? Why or why not?

b. Rewrite this code fragment showing what it would look like without using the variable i (but rather stepping through the array elements by incrementing an internal pointer). Note that this problem does not require you to know anything about how optimizers work. You are only being asked to show off your understanding of Squid to come up with a super-efficient Squid tuple sequence for a specific algorithm.

5. Suppose we have a compiler in which the ASTs for short-circuit or-expressions were modeled as binary expressions, instead of as a single node with two or more disjuncts.

   a. Draw the AST for `x || y || z` under this assumption.

   b. Write out the tuple sequence produced by a naive translation of this tree.

   c. Write out a more efficient sequence of tuples (Hint: only one temporary should be needed).

d. How would an optimizer detect that sequence is lousy (by looking only at the tuples)? What kind of transformations would an optimizer do (at the tuple level) to turn the lousy tuple sequence into the good one?

e. Explain how treating these operators an n-ary rather than binary, simplifies this issue a great deal. Use a tree grammar in your explanation.

Little Languages

1. Here is a cool little functional language:

```
PROGRAM →  (DECL ';')* EXPR
DECL    →  'val' ID '=' EXPR
        | fun ID '(' PARAMS? ')' '=' EXPR
EXPR    →  NUMLIT | ID | UOP EXPR | EXPR BOP EXPR
        | EXPR '?' EXPR ':' EXPR |  ID '(' ARGS? ')' | '(' EXPR ')'
PARAMS  →  ID (',' ID)*
ARGS    →  EXPR (',' EXPR)*
UOP     →  '-' | 'abs' | 'not'
BOP     →  '+' | '-' | '*' | '/' | 'mod' | 'and' | 'or' | '==' | '<'
```

a. Why is this called a functional language?

b. Is the grammar ambiguous? Why or why not?

c. Give a hierarchy of entity classes for this language.

d. Write a Greatest Common Divisor function in this language.

e. Give three examples of syntax errors and three examples of static semantic errors in this language. Make sure to write down all your assumptions; I did not give you any semantics so you will have to make up something reasonable.

2. Here is a small expression language:

```
Exp     →  Exp Exp op  | intlit
op      →  "+"  |  "-"  |  "*"  |  "/"
```

a. What language is this?

b. Is the grammar ambiguous? Why or why not?

c. Is it LL(k) for any k? If so, for which k? If not, why not?

d. Give a class hierarchy of entities for this language.

e. Give an attribute grammar for this language that can be used to evaluate expressions.

f. Give an attribute grammar for this language that attaches a "nesting level" to each identifier. You will have to make a slight modification to the original grammar for this to make sense.

3. This little language looks like an abstraction of something you might see in a real programming language. What, exactly? And is the grammar $LL(k)$?

```
G -> (S s G)?
S -> V q e | i f E g | V x
V -> i | V d i | V a E a
E -> n | V
```

4. Remove the left recursion from this grammar:

```
B -> (a|b)*A | bba*c
A -> Ac | d
```

5. Consider a language for describing vector graphics. An example program in this language (formatted ugly to highlight the fact that line breaks do not matter) is:

```
down deg color 1 0 0 left
90 forward 4 color 0 0 1 [ left 90
forward 1.5 ] right 90 forward 1.5 up
```

This program draws the letter T with a red vertical line of size 4 units and topped with a 3 unit blue line. A program is a sequence of instructions. The instructions are:

Instruction	**Description**
`deg`	switch to degree mode
`rad`	switch to radians mode
`down`	put the pen down so movements draw lines
`up`	pick the pen up so movements don't draw anything
`left θ`	turn counterclockwise by angle `θ`
`right θ`	turn clockwise by angle `θ`
`forward n`	draw a line by moving forward n units.
`backward n`	draw a line by moving backward n units.
`color r g b`	set color (r,g,b), values are floats in the range 0 to 1.
`[`	save current state
`]`	restore previously saved state

Give an Ohm grammar for this. Also answer: is it even possible to give an unambiguous CFG for this language? Why or why not?